

Comparaison de deux méthodes de recherche de mots en C++

Projet *Find* : `string::find` contre parcours de `vector<char>`

Elmachraa Doha
ILISI2
Programmation Object avantee

04/10/2026

Table des matières

1	Introduction	2
2	Cahier des charges et organisation	2
2.1	Fonctionnalités demandées	2
2.2	Organisation des fichiers	2
3	Conception et implémentation	2
3.1	La classe Find1	2
3.2	La classe Find2	2
3.3	Une différence de sémantique à retenir	3
4	Validation	3
5	Méthode de mesure	3
5.1	Programme principal	3
5.2	Conditions d'exécution	4
6	Résultats	4
6.1	Mesure du programme initial	4
6.2	Mesure équitable	4
7	Analyse	5
8	Améliorations possibles	5
9	Conclusion	5

1 Introduction

Ce projet consiste à écrire deux classes C++ qui répondent à la même question : « le mot `mot2` se trouve-t-il dans le mot `mot1` ? » Les deux classes diffèrent par la représentation des données et par l'algorithme utilisé. L'objectif est de les implémenter, de vérifier leur fonctionnement, puis de comparer leurs temps d'exécution sur un fichier de 10 000 mots.

Le rapport présente d'abord la conception, puis la méthode de mesure, les résultats, et enfin une analyse critique de cette mesure.

2 Cahier des charges et organisation

2.1 Fonctionnalités demandées

- Une classe `Find1` qui stocke deux `string` et cherche `mot2` dans `mot1`.
- Une classe `Find2` qui stocke deux `vector<char>` et effectue la même recherche avec une boucle écrite à la main.
- Un programme principal qui lit 10 000 mots dans `mots.txt`, demande `mot2` à l'utilisateur et mesure le temps de chaque classe pour 10, 100, 1 000 et 10 000 mots, chaque test étant répété 10 000 fois.

2.2 Organisation des fichiers

Le projet est développé avec Code : :Blocks et le compilateur GCC. Les déclarations sont séparées des définitions.

Fichier	Rôle
<code>include/Find1.h, src/Find1.cpp</code>	Classe <code>Find1</code> (<code>string</code>)
<code>include/Find2.h, src/Find2.cpp</code>	Classe <code>Find2</code> (<code>vector<char></code>)
<code>main.cpp</code>	Lecture du fichier et mesure des temps
<code>test.cpp</code>	Test unitaire simple des deux classes
<code>mots.txt</code>	10 000 mots utilisés pour les mesures

TABLE 1 – Structure du projet

3 Conception et implémentation

3.1 La classe `Find1`

`Find1` utilise la fonction `find` de la bibliothèque standard. Elle recherche `mot2` comme une *sous-chaîne* de `mot1` : les lettres doivent être consécutives.

```
1 bool Find1::Chercher() const
2 {
3     return mot1.find(mot2) != string::npos;
4 }
```

3.2 La classe `Find2`

`Find2` parcourt `mot1` une seule fois avec un indice `j` sur `mot2`. Chaque fois que la lettre courante de `mot1` est égale à `mot2[j]`, on avance `j`. Si `j` atteint la taille de `mot2`, la recherche réussit. Elle recherche donc `mot2` comme une *sous-séquence* : les lettres doivent apparaître dans l'ordre, mais pas forcément côte à côte.

```

1 bool Find2::Chercher() const
2 {
3     if (mot2.empty())
4         return true;
5
6     int j = 0;
7     for (int i = 0; i < mot1.size(); i++)
8     {
9         if (mot1[i] == mot2[j])
10        {
11            j++;
12            if (j == mot2.size())
13                return true;
14        }
15    }
16    return false;
17 }

```

Complexité. Pour un mot `mot1` de longueur n et un motif `mot2` de longueur m , `Find2` est en $O(n)$ dans le pire cas. L'algorithme glouton est correct : avancer dès qu'une lettre correspond ne peut pas faire manquer une sous-séquence existante. L'implémentation de `string::find` est en $O(n \cdot m)$ dans le pire cas théorique, mais très rapide en pratique pour des mots courts.

3.3 Une différence de sémantique à retenir

Les deux classes ne résolvent *pas exactement* le même problème. Avec `mot1 = "compteur"` et `mot2 = "cte"` :

- `Find1` répond **non**, car « cte » n'apparaît pas d'un seul bloc ;
- `Find2` répond **oui**, car c, puis t, puis e apparaissent dans cet ordre.

Tout mot qui contient `mot2` comme sous-chaîne le contient aussi comme sous-séquence, mais pas l'inverse. Le nombre de réussites de `Find2` est donc toujours supérieur ou égal à celui de `Find1`.

4 Validation

Le programme `test.cpp` vérifie les deux classes sur l'exemple ci-dessus. Son exécution affiche :

```

Find1 : mot2 n'existe pas dans mot1
Find2 : mot2 existe dans mot1

```

Ce résultat est conforme à l'analyse précédente. Le cas `mot2` vide est traité dans `Find2` (réponse vraie) et, dans `Find1`, `find` d'une chaîne vide renvoie la position 0.

5 Méthode de mesure

5.1 Programme principal

`main.cpp` lit les mots, demande `mot2`, puis pour chaque taille $n \in \{10, 100, 1000, 10000\}$ effectue $n \times 10\,000$ recherches avec chacune des classes. Le temps est mesuré avec `high_resolution_clock` (de `std::chrono`) et on en déduit le temps moyen par recherche.

5.2 Conditions d'exécution

Les résultats ci-dessous ont été obtenus avec GCC 13.3 sous Linux, avec `mot2 = "cte"`. Ils dépendent de la machine : **à remplacer par vos propres valeurs si vous relancez le programme.**

6 Résultats

6.1 Mesure du programme initial

Le tableau donne le temps total en millisecondes pour $n \times 10\,000$ recherches, avec le code du projet tel qu'il est écrit.

n	Mode Debug (-g)		Mode Release (-O2)	
	Find1 (ms)	Find2 (ms)	Find1 (ms)	Find2 (ms)
10	6	89	4	13
100	78	1 024	52	113
1 000	831	9 509	562	1 416
10 000	8 826	98 293	6 286	14 095

TABLE 2 – Temps totaux du programme `main.cpp`

Deux constats : le temps total est proportionnel à n (le temps moyen par recherche reste constant), et `Find1` semble plus rapide, d'un facteur environ 10 en Debug et 2 à 3 en Release. L'optimisation du compilateur divise le temps de `Find2` par 7 environ, ce qui montre l'importance de mesurer en mode Release.

6.2 Mesure équitable

Dans le programme initial, la mesure de `Find2` inclut la création de deux `vector<char>` à partir des chaînes et la copie des paramètres par valeur à chaque itération. `Find1` copie aussi ses chaînes, mais les petites chaînes sont stockées dans l'objet lui-même et évitent l'allocation mémoire. On compare donc autre chose que les algorithmes.

J'ai refait la mesure avec les vecteurs construits une seule fois avant le chronomètre, des paramètres passés par référence constante, une compilation en `-O2`, et un chronométrage en nanosecondes. Les résultats (temps moyen par recherche) sont :

n	Find1 (ns)	Find2 (ns)	Réussites Find1	Réussites Find2
10	9,1	7,6	2	3
100	9,7	5,7	11	30
1 000	8,9	5,6	78	261
10 000	19,9	26,6	763	2 602

TABLE 3 – Mesure équitable, mode Release, `mot2 = "cte"`

Les colonnes de droite donnent le nombre de mots où `mot2` est trouvé. Elles confirment la différence de sémantique : `Find2` trouve nettement plus de mots que `Find1`.

7 Analyse

1. **L'écart du tableau 2 est surtout dû à la mesure**, pas aux algorithmes. Une fois les copies et conversions retirées (tableau 3), les deux méthodes sont du même ordre de grandeur, quelques nanosecondes par mot.
2. Pour 10 à 1000 mots, `Find2` est un peu plus rapide ; pour 10 000 mots, il devient plus lent. Une explication plausible est l'effet du cache mémoire : les 10 000 petits vecteurs sont alloués séparément sur le tas, alors que les chaînes courtes sont stockées directement dans l'objet. Cette explication n'a pas été vérifiée par un profileur.
3. Le mode **Release** apporte un gain bien plus important que le choix entre les deux classes.
4. Les deux classes **ne répondent pas à la même question** (sous-chaîne ou sous-séquence), donc la comparaison de vitesse n'a de sens qu'avec cette réserve.

8 Améliorations possibles

- Passer les paramètres par référence constante (`const string&`, `const vector<char>&`).
- Remplacer `int` par `size_t` pour supprimer les avertissements `-Wsign-compare`.
- Retirer `using namespace std` des fichiers d'en-tête.
- Supprimer le `virtual` des destructeurs, inutile sans héritage.
- Afficher les compteurs de réussites et vérifier que la saisie de `mot2` a réussi.
- Ajouter des tests : `mot2` vide, plus long que `mot1`, lettres répétées.

9 Conclusion

Les deux classes fonctionnent et répondent à deux variantes d'un même problème. La première mesure donnait l'impression que `string::find` était très supérieur, mais une mesure équitable montre que les deux approches ont des performances proches. Ce projet illustre une règle générale : **il faut mesurer correctement avant de conclure**, en compilant en mode Release, en préparant les données hors du chronomètre et en comparant des traitements équivalents.